Compute Model Recommendation

Crestline Regional Services: Reporting and Lookup Workload

Prepared for	Crestline Regional Services (engagement requested by HarborTech)
Prepared by	Cloud Operations Analyst
Date	October 13, 2026
Course / Assignment	IT 4560, Week 06: Containers and Serverless
Playbook entry	week06-containers-serverless

Note on figures. The workload details, traffic numbers, and costs in this document are invented sample values, chosen to make the analysis concrete. Swap in measured values from Crestline's CloudWatch and billing data before relying on any of them for a real decision.

1. Recommendation

Move to a combination of managed services, with serverless as the default execution model. Retire the always-on EC2 instance after a parallel-run validation period.

Component	Recommended service	Why
Nightly reporting task	Amazon EventBridge Scheduler triggering AWS Lambda	Runs about 4.5 minutes once a day. Paying for 24 hours to cover 4.5 minutes is the core mismatch.
Lookup API (3 endpoints)	Amazon API Gateway (HTTP API) in front of AWS Lambda	About 1,500 requests per day, concentrated in business hours, near zero overnight.
Fallback if the report outgrows Lambda	EventBridge Scheduler triggering an ECS on Fargate scheduled task	Same trigger, no 15-minute limit, more CPU and memory available.
Current EC2 instance	Stop after a 2-week parallel run, terminate after a 30-day rollback window	Removes 24/7 cost, OS patching, and single-instance risk.

In one sentence: the workload is intermittent, short, and trigger-driven, so it should run only when something triggers it, and AWS rather than Crestline should own the operating system underneath.

2. Workload Profile (sample values)

These are the facts the recommendation rests on. If measured values differ, see Section 9.

Attribute	Current value
Hosting	One EC2 t3.medium (2 vCPU, 4 GB RAM), Amazon Linux 2023, us-east-2, running 24/7
Application	Python 3.12 application, three REST endpoints plus one reporting script run by cron
Key libraries	pandas, psycopg2, boto3, Jinja2 (reporting); Flask and psycopg2 (API)
Database	Amazon RDS for PostgreSQL (db.t3.small), unchanged by this project
Nightly report	Daily Service Request Summary, starts 2:00 AM Central, reads about 180,000 rows, writes CSV and PDF to S3, emails 12 recipients through SES
Report runtime	About 4 min 30 s typical, 6 min worst case last quarter; peak memory about 900 MB
API endpoints	GET /requests/{id}, GET /requests?status=, POST /requests/{id}/notes

Attribute	Current value
API traffic	About 1,500 requests/day; peak about 20 requests/minute from 8:00 to 10:00 AM; near zero 8 PM to 6 AM
API latency	Median 120 ms, 95th percentile 310 ms
Instance utilization	Average CPU 2%, peak 9%; average memory use 31%
Local state	None. Logs written to local disk and rotated; no durable data on the instance
Operations team	Two cloud operations staff; patch window the first Sunday of each month

3. Evaluation Against the Assignment Criteria

Criterion	What the workload shows	Implication
How often it runs	Report: once per day. API: about 1 request per minute on average	Long idle gaps. Always-on capacity is mostly unused.
Needs a running OS?	No. Both pieces start in response to a clock or an HTTP request	No technical reason to keep an OS running between triggers.
Scaling	Peaks of about 20 requests/minute, then nothing. Report is one job	Needs scale-to-zero and automatic scale-out, not a fixed instance size.
Administrative responsibility	Crestline owns OS, runtime, patching, cron, TLS, and instance recovery	Highest responsibility for the lowest-value work.
Patching and maintenance	Monthly patch window with a reboot; about 4 staff-hours per month	Recurring toil that serverless shifts to AWS.
Dependencies	Standard Python libraries, no OS-level packages, no custom agents	Fits Lambda. Container image available if dependencies grow.
Execution duration	Report 4.5 to 6 minutes; API calls under half a second	Report has about 2.5x headroom under Lambda's 15-minute limit.
Cost and utilization	About $33/month for an instance at 2% average CPU	Pay-per-use billing matches the usage pattern (Section 6).

4. Comparing the Options

Remain on EC2 (virtual machine)

Full control of the OS and no application changes, and it suits software that needs OS-level customization, long-running processes, or steady high utilization. For this workload it means paying for 24/7 capacity at 2% CPU, owning patching and hardening, and relying on one instance with no failover. The report's cron job also stops silently if the instance does. **Verdict:** justified only by hard OS-level dependencies, which this application does not have.

Containers (the concept)

A container packages the code and its dependencies into an image that runs the same way everywhere. That makes dependencies explicit and deployments repeatable. It is a *packaging* decision, not an *operating* decision: something still has to run the container, and that choice determines the real administrative burden. **Verdict:** useful as a format (including as a Lambda deployment option), but not a reason to move by itself.

Amazon ECS

On EC2: Crestline would still patch and scale the underlying hosts, adding a cluster, task definitions, and a load balancer to what is today one server. That is more complex than the status quo. **On Fargate:** AWS manages the hosts and Crestline pays only while tasks run. An always-on API service needs at least one running task (about $15 to $20/month for a small task plus an Application Load Balancer at roughly $17/month), which is still far above the serverless estimate. **Verdict:** ECS on EC2 is overkill. Fargate is the right fallback for the report if it outgrows Lambda.

AWS Lambda

No servers to patch, automatic scaling from zero, and billing per invocation and per millisecond. Limits to design around: 15-minute maximum duration, cold starts after idle periods, memory and package-size ceilings, and careful database connection handling. All are manageable here (Section 8). **Verdict:** best fit for both the report and the API.

Amazon API Gateway

A managed front door that handles routing, TLS, throttling, authorization, and access logs, then invokes Lambda. An **HTTP API** is the simpler and cheaper choice for three endpoints; a REST API is only needed for features such as usage plans, API keys, or request validation. The current API key header check would be replaced with a JWT authorizer backed by an Amazon Cognito user pool. **Verdict:** replaces the web server, reverse proxy, and certificate management on the instance.

Event-driven execution

Something other than a person or a running server decides when work begins. EventBridge Scheduler replaces cron, with built-in retries and a dead-letter queue, and the same pattern extends to S3 uploads or queue messages if reporting grows (for example, triggering a report when a data file lands in a bucket). **Verdict:** replaces the scheduler and adds reliability and visibility that a crontab does not have.

5. Decision Matrix

Scores run from 1 (poor fit) to 5 (strong fit) for this workload, using the profile in Section 2.

Factor	EC2	ECS on EC2	ECS on Fargate	Lambda + API GW + EventBridge
Matches intermittent usage	1	1	3	5
Cost at low utilization	1	1	3	5
Patching and admin burden	1	1	4	5
Automatic scaling	2	3	4	5
Handles heavy or unusual dependencies	5	4	4	3
Handles long-running jobs	5	5	5	2
Migration effort (5 = easiest)	5	2	3	3
Resilience (no single server)	2	3	4	5
Total (out of 40)	**22**	**20**	**30**	**33**

Lambda leads on the factors that drive this decision (utilization, cost, maintenance, resilience). EC2 and Fargate lead only on dependency flexibility and long-running jobs, neither of which this workload needs today.

6. Cost and Resource Utilization (illustrative monthly estimates)

Line item	Current (EC2)	Recommended (serverless)
Compute	t3.medium on-demand, about $30	Report: 30 runs x 270 s x 1.5 GB is about 12,150 GB-seconds, about $0.20. API: about 45,000 invocations, under $0.50 (often inside the free tier)
Storage	30 GB gp3 volume, about $2.40	None needed (code is deployed as a package)
API front end	Included in instance (self-managed)	HTTP API at about $1 per million requests, so about $0.05
Scheduler	Cron (free, but unmonitored)	EventBridge Scheduler, effectively $0
Logs and alarms	About $1 (CloudWatch agent)	About $1 to $2 (CloudWatch Logs and alarms)
Staff time (patching)	About 4 hours/month	About 0.5 hours/month (runtime version updates)
Approximate total (excluding RDS)	**about $33/month**	**about $3/month**

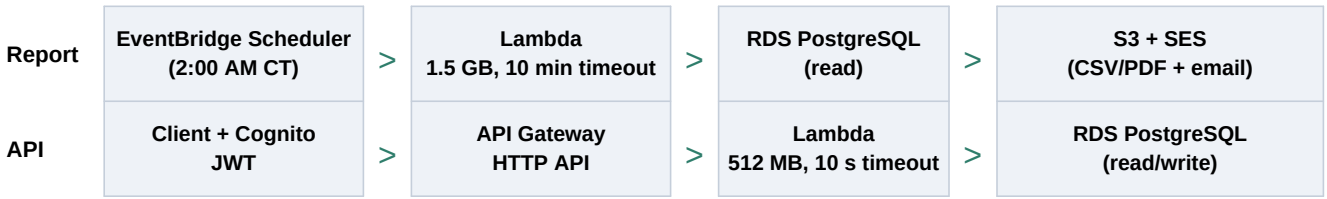
Utilization: the instance averages 2% CPU, so about 98% of the paid capacity is idle. At these volumes the serverless design is roughly 90% cheaper on infrastructure, and the larger saving is the roughly 3.5 staff-hours per month no longer spent on patch windows. RDS cost is the same under both designs and is excluded.

7. Operational Responsibility and Target Architecture

Responsibility	EC2 today	Recommended design
OS patching and hardening	Crestline	AWS
Runtime (Python) patching	Crestline	AWS maintains the runtime; Crestline bumps the version setting and redeploys
Capacity planning	Crestline	AWS (automatic scaling)
Instance health and recovery	Crestline	AWS
Scheduler uptime	Crestline (cron on the instance)	AWS (EventBridge Scheduler)
TLS, routing, throttling	Crestline	AWS (API Gateway)
Application code and dependencies	Crestline	Crestline
IAM permissions, secrets, data security	Crestline	Crestline
Monitoring and alerting	Crestline	Crestline (CloudWatch metrics, logs, alarms)

Serverless does not remove operational work; it removes the repetitive, patch-heavy layers. Crestline keeps ownership of the code, permissions, and data, which is the shared responsibility model working as designed.

Target request flow



The report and the API are separate functions with separate IAM roles, so the API cannot send email and the report cannot be called from the internet. The API function leaves pandas out of its package, which keeps its cold start to roughly half a second instead of two seconds or more.

8. Risks and Mitigations

Risk	Mitigation
Cold starts add latency to the first API call after idle (about 0.5 s expected for the lean API package)	Acceptable for an internal lookup tool. If it becomes a problem, enable provisioned concurrency for the API function.
Report outgrows 15 minutes (today 4.5 to 6 min)	Alarm if any run exceeds 10 minutes. Move the report to an ECS Fargate scheduled task; the EventBridge trigger stays the same.
Database connection exhaustion from concurrent Lambda invocations	Cap API function concurrency at 20 and reuse connections across invocations. Add RDS Proxy if traffic grows well beyond today's levels.
Dependencies too large for a zip package	Package as a container image (up to 10 GB) for Lambda; the code does not change.
VPC networking needed to reach private RDS adds complexity	Place the functions in private subnets with a security group allowed to the database; use VPC endpoints for S3 and Secrets Manager to avoid NAT cost.
Failed scheduled run goes unnoticed	Configure retries, a dead-letter queue, and a CloudWatch alarm on errors that notifies the on-call email list.
Runaway invocations or abuse drive up cost	API Gateway throttling, function concurrency limits, and an AWS Budgets alert at $15/month.
Hidden dependency on the instance (local files, cron entries, agents)	Complete the inventory step in Section 10 before building.

9. When I Would Choose Differently

- **Stay on EC2** if the application needs specific OS packages, a licensed agent, a persistent local filesystem, or a long-lived background process and a rewrite is not justified. In that case, right-size to a t3.small, automate patching with Systems Manager, and stop the instance outside business hours.
- **Use ECS on Fargate for both pieces** if the team already standardizes on containers, the dependencies become complex, or the report regularly exceeds 10 minutes.
- **Keep the report on Lambda but move the API to Fargate** if API traffic grows to steady, sustained load (roughly tens of millions of requests per month) where always-on containers become cheaper and more predictable.
- **Reconsider the whole design** if measured runtime, traffic, or dependencies differ materially from Section 2.

The recommendation is a default, not a dogma: the compute model should follow how the workload behaves, and this workload is sporadic, short, and event-triggered.

10. Migration Plan (4 weeks)

Week	Activity	Exit criteria
1	Inventory the instance: code, Python version, installed packages, cron entries, environment variables, outbound network access, IAM role. Split the report and API into separate codebases.	Written inventory; no unexplained local dependencies
2	Move secrets to Secrets Manager and any local files to S3. Build both Lambda functions, the HTTP API, Cognito authorizer, and EventBridge schedule as infrastructure as code (AWS SAM). Add logs, alarms, and a dead-letter queue.	Deploys cleanly to a test account from a single command
3	Parallel run: new report writes to a test S3 prefix and emails two testers; compare output to the EC2 report for 7 nights. Route a test client and a small group of users to the new API.	7 consecutive matching reports; API error rate under 0.5%; p95 latency under 500 ms
4	Cut over: point clients and DNS to API Gateway, disable cron on the instance, enable the production schedule. Stop (do not terminate) the instance.	No incidents for 5 business days
+30 days	Snapshot the volume, terminate the instance, remove the patch window from the operations calendar.	Rollback window closed

11. Conclusion

The application works, but the operating model no longer fits it. An always-on virtual machine buys continuous availability and OS-level control that this workload does not need, and charges for it in cost, patching effort, and single-instance risk.

The best fit is a **combination of managed services**: **EventBridge Scheduler and Lambda** for the nightly report, **API Gateway and Lambda** for the API, and **ECS on Fargate** held in reserve for anything that exceeds Lambda's limits. This design runs code only when it is needed, cuts the sample infrastructure bill from about $33 to about $3 per month, frees roughly 3.5 staff-hours per month, and leaves Crestline responsible only for what only Crestline can own: the code, the permissions, and the data.